目录

引言 **1**

全书结构 2

如何阅读本书 4

前置知识 4

致谢 5

引言

2025 年 8 月至 10 月，我在图灵《AI Agent 实战营》中讲授了一系列课程。课程围绕一个核心问题展开：如何以可解释、可验证、可复用的工程原则设计 AI Agent，而不是依赖经验和直觉反复试错？因此，课程不以跑通一个演示系统为终点，而是进一步分析 Agent 为何采用特定架构，以及每项设计决策对应的能力边界与工程取舍。本书以课程讲义和配套实验为基础，经过系统整理、补充与重构而成。

本书的形成过程本身也是一次人机协作实践。从最初构思到书稿定型，我主要采用 **whisper coding**（口述式协作）的方式，与 Pine 的语音 Agent 共同完成资料整理和文本迭代。准备课程时，我先口述章节提纲，由 Agent 调研相关工作并形成材料草稿；课程结束后，再结合学员反馈，对论证、案例和实验反复核查与修订。语音降低了将想法外化为文本的成本，也缩短了“提出问题—检索资料—形成草稿—复核修改”的迭代周期。因而，这本书不仅讨论 Agent，也记录了一种由 Agent 深度参与的知识生产方式。

自 2025 年初 DeepSeek R1 发布以来，AI 研究与产业实践的重心逐步从基座模型能力扩展到 Agent 的系统化落地。在模型侧，智能体强化学习（Agentic Reinforcement Learning）开始将工具调用及环境交互能力内化到模型参数中，使模型在编程（coding）、数学推理和图形界面操作（computer use）等领域表现出更强的通用能力。在产品侧，Manus、Claude Code、OpenClaw 等通用 Agent 则推动了人机交互方式的变化，并使“代码生成 + 文件系统”成为重要的系统架构范式。

重新审视课程中总结的 Agent 架构原则，可以发现：尽管模型和产品快速迭代，这些原则仍然具有稳定的解释力。业界后来广泛使用 Skill、harness、loop engineering 等术语，但相应的工程实践往往更早出现；概念的形成，是对大量实践经验的归纳，而不是实践的起点。换言之，实践在前，命名在后。

这些原则主要来自长流程、高风险场景中的工程实践。作为 Pine AI 的首席科学家，我与团队共同研发了 Pine，使 Agent 能够代表用户与真人沟通，并处理账单协商、退款投诉和订阅取消等涉及真实经济利益的复杂任务。这类任务通常跨越较长时间、包含多轮交涉，且单个步骤的错误就可能造成实际损失。对可靠性、安全性和可审计性的严格要求，促使我们逐步形成了本书所强调的架构原则。下面几个例子均来自这段实践。

- 早在 Skill 概念流行之前，我们就已经采用动态加载提示词的方法解决提示词无限膨胀的问题，采用命令行执行工具的方式解决工具列表无限膨胀的问题，采用系统状态栏技术解决 Agent 不感知执行环境和用户时间、工作状态等问题。
- 早在 harness 概念流行之前，我们就已经采用类似 Claude Code 的方法，解决模型工具调用不稳定、产生幻觉、执行危险或越权操作、不遵循指令等问题。
- 早在 loop engineering 概念流行之前，我们就在使用本书称为提议者-审核者（proposer-reviewer）的方法解决模型过早认为任务完成的问题。

这些方法并非 Pine 所独有。许多模型与 Agent 团队在解决相似问题时，都独立形成了相近的工程方法；这种趋同说明，它们反映的是 Agent 系统的共性约束。基于这些认识，我于 2025 年在图灵开设《AI Agent 实战营》，并于 2024—2026 年持续在中国科学院大学讲授 AI Agent 实践课程。本书选择开源发布，也希望让这些可复用的知识和经验服务于更多研究者与工程实践者。

实践在前，命名在后。对企业级 Agent 开发而言，这意味着实践不能始终滞后于概念的普及：当一个术语成为行业共识时，其对应的问题往往已经在领先系统中经过了长期探索。要更早识别并解决这些问题，我认为有两个关键条件。

第一，选择对 Agent 能力上限提出足够高要求的真实业务，并持续获取真实反馈。以 Pine 为例，一项任务可能持续数小时乃至数周，期间需要与多个利益相关方反复沟通，完成长时间通话、复杂表单操作和多轮邮件往来；同时还必须保证关键数字准确、操作可控，并始终维护用户利益。足够复杂的场景会持续暴露模型能力与业务要求之间的差距，从而促使团队构建 harness，系统解决模型暂时无法独立处理、业务却必须完成的问题。如果业务需求很容易被下一次模型升级覆盖，团队也较难积累稳定的系统设计原则。

第二，建立系统的评估（Evaluation）机制。没有评估，就无法判断一次改动带来的提升究竟源于设计本身，还是仅仅来自随机波动。评估把 Agent 的迭代从经验判断转化为可比较、可复现的工程过程，是以科学方法开展系统开发的基础。第七章将专门讨论这套方法。

不管底层模型如何升级，不管产品形态如何创新，几乎所有成功的 Agent 系统都遵循着相同的架构模式。这并非巧合：**好的设计原**

则本就应该穿越模型的迭代周期，因为它们描述的不是某个模型的用法，而是智能系统与世界交互的基本模式。

图灵奖得主、强化学习之父 Richard Sutton 曾说，宇宙演化经历了从尘埃到恒星、从恒星到生命、从生命到智能体（原文为设计实体，designed entities）的 4 个阶段。生物进化是盲目的：随机变异，自然选择。大多数生物并不理解自身的运作原理，也无法自主设计和改造自己。而智能体（Agent）是宇宙演化史上一种全新的存在：它能通过生成代码实现自举（bootstrap）和自我进化，就像一个程序员编写了另一个程序员，然后新的程序员又能继续编写下一个。也就是说，Agent 能够理解自身的运作机制，并根据目标创造全新的智能体，甚至改进自己。本书的使命，就是帮助你理解和掌握这种创造的原则。

本书的核心公式只有一句话：**Agent = LLM + 上下文 + 工具**。三者缺一不可。

更直观地说，就是**大脑 + 眼睛 + 手脚**。大脑（LLM）负责思考和决策，眼睛（上下文）决定 Agent 能看到什么信息，手脚（工具）决定 Agent 能做什么事情。（严格来说，“眼睛”只是一个粗略的类比：上下文不仅包含环境信息和对话历史，还包含工具定义等内容，也就是说 Agent “看到”的信息中也包括了“有哪些手脚可用”。这个隐喻旨在传达核心直觉：上下文是模型能感知到的一切信息。）

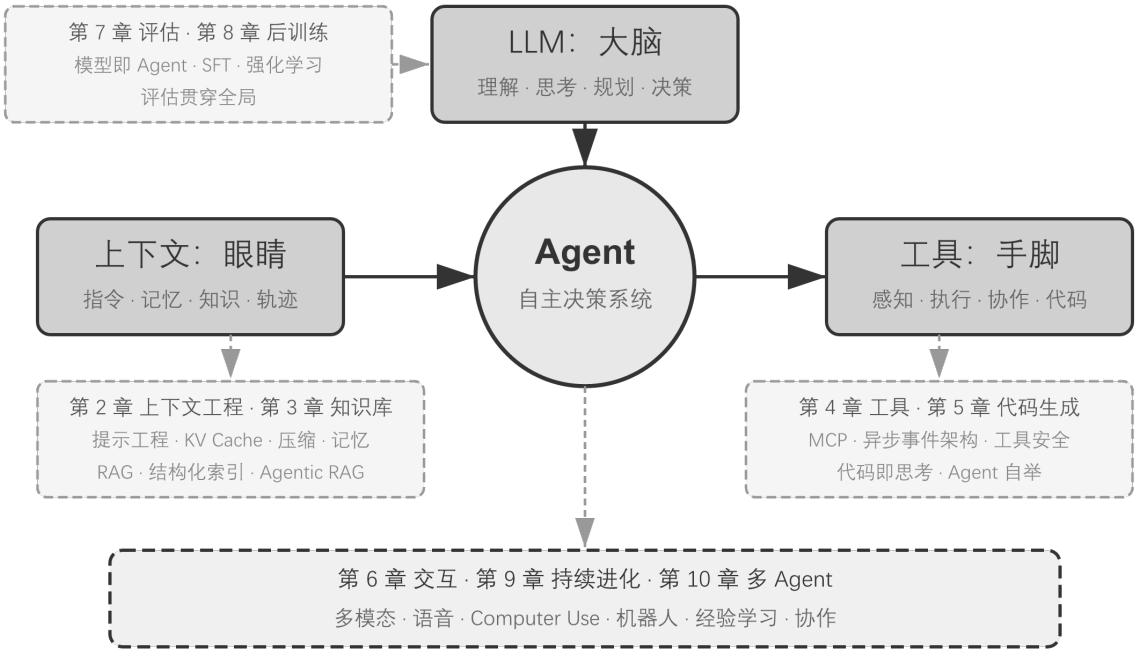


图 0-1 Agent = LLM + 上下文 + 工具

对熟悉强化学习的读者，这三者也可以映射到 RL 的形式化语言。具体来说，LLM 对应 Policy（策略），上下文对应 Observation Space（观察空间），工具对应 Action Space（动作空间）。三种说法对应同一个对象，只是表达层次不同。

全书结构

本书共十章，围绕两个相互衔接的问题组织（图 0-2）。**第一部分“如何构建 Agent”**包括第一至六章：从基本概念出发，依次讨论上下文、记忆与知识、工具、Coding Agent，以及观察与动作空间的扩展。**第二部分“如何提升 Agent 能力”**包括第七至十章：先用评估建立可度量的反馈，再通过模型后训练、基于运行经验的持续进化和多 Agent 协作，分别从模型参数、单体系统和群体系统三个层面扩展能力。

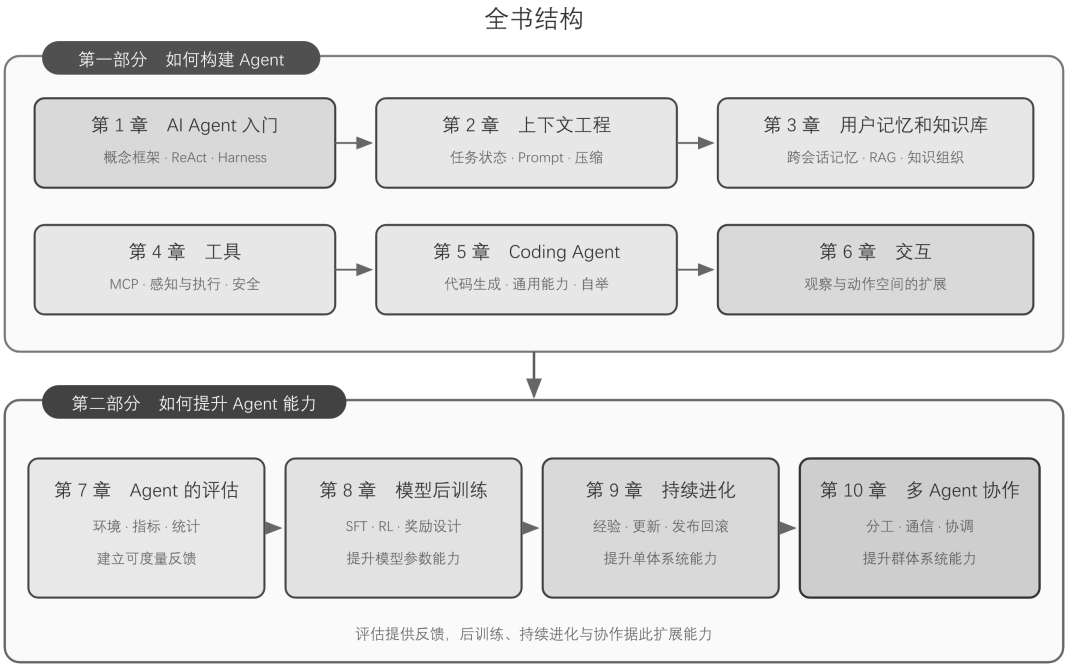


图 0-2 全书结构：构建 Agent 与提升 Agent 能力

- **第一章（AI Agent 入门）**从多个真实 Agent 产品出发，建立对 Agent 的直观理解。深入解析 Agent 的核心公式：从实现层的 LLM + 上下文 + 工具，到直觉层的大脑 + 眼睛 + 手脚，再到学术层的策略 (Policy)、观察空间 (Observation Space) 与动作空间 (Action Space)。同时通过实验剖析 ReAct 循环的运作机制，也就是“思考 → 行动 → 观察”的迭代过程，并区分任务内的上下文适应、跨任务的外部产物 (artifact) 更新和训练周期中的参数更新。最后讨论从工作流到自主 Agent 的编排设计模式，为后续章节建立统一的概念框架。
- **第二章（上下文工程）**是全书最关键的一章，系统讲解上下文，也就是 Agent 的“眼睛”。本章先从 API 消息结构与 Agent 核心循环讲起，为“上下文就是消息列表”这一认识奠定基础，再深入 KV Cache（大模型推理过程中复用历史计算结果的机制）的底层原理，然后依次展开：提示工程 (Prompt Engineering，包括流程化设计、工具描述、业务规则细化) 与提示注入 (Prompt Injection) 攻防、Agent Skills 的按需加载机制、Agent 状态栏技术，以及上下文压缩 (Context Compression) 策略。各术语的完整定义在正文首次出现处给出。
- **第三章（用户记忆和知识库）**将上下文管理延伸到跨会话的持久化知识体系，让 Agent 不仅能记住当前对话的内容，还能在多次对话间积累和调用知识。涵盖用户记忆的四种渐进式策略、RAG（检索增强生成，即先检索相关文档再让模型生成回答）的完整技术栈（包括不同的文本搜索方法和搜索结果排序优化）、更高级的知识组织方法、智能体化 RAG（Agentic RAG，即让 Agent 自主决定何时检索、检索什么），以及多模态记忆的前沿探索。
- **第四章（工具）**探讨 Agent 与外部世界交互的桥梁：工具就像是 Agent 的“手脚”，让它能够搜索网页、调用 API、操作数据库等。介绍 MCP 工具互操作标准和五类工具的设计原则（感知、执行、协作、事件触发、用户沟通），展开多模态感知的三条技术路线（原生多模态处理、提取为文本、工具化多模态分析），并重点阐述执行工具的安全机制；事件触发、用户沟通与异步运行时则在第六章展开。
- **第五章（Coding Agent 与通用 Agent）**论证了 Coding Agent 加上文件系统，是所有通用 Agent 最核心的技术基础。以 OpenClaw 架构为主线，剖析 Coding Agent 的工作流程和实现技巧，并展示代码生成在编程之外的广泛价值：从辅助思考、构建知识库，到动态创造新工具和 Agent 自举。
- **第六章（交互：观察与动作空间的扩展）**从**模态**与**时序**两个维度研究 Agent 与环境的交互机制：异步与事件驱动使 Agent 能够持续接收外部事件并及时响应；语音交互将感知、推理与生成推进到实时尺度；Computer Use 将动作空间扩展到图形界面；机器人操作进一步把动作延伸到物理环境。四类场景共享唤醒、安全点、取消、抢占与快慢路径分离等系统原语。
- **第七章（Agent 的评估）**构建一套科学的评估方法论。覆盖评估环境（工具调用型和人机交互型两种核心范式，以及章末单独讨论的仿真环境）、数据集的设计原则、LLM-as-a-Judge 自动化评判方法、评估驱动的模型选型，以及将评估结果转化为系统改进的

完整闭环。

- **第八章（模型后训练）**深入 SFT（监督微调，即用标注数据教模型“照样学样”）与 RL（强化学习，即让模型通过试错和奖励反馈自主提升）这两种后训练技术。以“SFT 记忆、RL 泛化”和“数据与环境比算法更重要”为核心论点，涵盖预训练/SFT/RL 三阶段全景、经典 RL 理论、奖励信号设计（从二元奖励到过程奖励、再到“奖励结果、约束过程”的验证路径惩罚）、单轮与多轮强化学习算法，以及样本效率优化等前沿探索。
- **第九章（Agent 的持续进化）**研究如何把 Agent 的运行经验转化为下一版本的能力。全章先建立由环境结果、过程规则和 LLM Rubric 组成的学习信号，再比较知识文档、Prompt 与 Skills、程序与 Harness、模型参数四种更新载体，最后讨论新版本如何经过验证、灰度发布、回滚与长期整理。
- **第十章（多 Agent 协作）**从单体能力扩展到系统级协同，讨论多个 Agent 如何分工合作。系统阐述多 Agent 协作的分类框架（上下文共享/独立 × 对等/管理者/去中心化），通过翻译 Agent、电话与电脑协同 Agent 等案例展示协作架构的设计方法，并讨论 Agent 社会和 Agent 经济的潜在发展方向。

如何阅读本书

本书的各章节相对独立，你可以根据自己的需求选择不同的阅读路径：

- **如果你是 Agent 开发者**，建议按顺序阅读全书：第一至六章建立完整的 Agent 构建方法，第七至十章则从评估、模型后训练、持续进化和多 Agent 协作四个方向讨论能力提升。
- **如果你时间有限**，优先阅读第一章（建立全局认知）和第二章（掌握最关键的上下文工程）。第二章中 KV Cache 的底层原理较为技术化，初次阅读可先跳过原理部分、只记住开头给出的三条核心结论，不影响后续理解。
- **如果你关注模型训练**，可以直接阅读第八章（模型后训练）；其中评估方法（第七章）是训练的前提，建议一并阅读，并先读第一至二章以建立整体认知。

每章都包含大量的**实验**和**思考题**，编号格式为“实验 X-Y”（X 为章节号，Y 为章节内序号）。实验和思考题的标题中用星级标注难度：★ 表示入门级，适合所有读者；★★ 表示中等难度，需要一定的工程实践基础；★★★ 表示进阶挑战，通常涉及开放性问题或复杂的系统设计。大部分实验配有完整的可运行代码，组织在配套的开源仓库中：

> **配套代码仓库**：https://github.com/bojieli/ai-agent-book

可以使用 Git 获取全部配套代码：

```
git clone https://github.com/bojieli/ai-agent-book.git
cd ai-agent-book
```

不熟悉 Git 的读者也可以在仓库页面点击 **Code → Download ZIP** 下载。实验代码按章节组织在 `chapter1/` 至 `chapter10/` 中；要查找“实验 X-Y”，请先打开对应的 `chapterX/README.md`，根据实验编号找到项目目录，再按照项目自身的 README 安装依赖并运行。部分标为“复现指南”的实验依赖外部仓库，具体获取方式也会在相应的 README 中说明。

我强烈建议你动手跑一遍这些实验。AI Agent 是一个实践性极强的领域，很多设计上的直觉需要在动手调试的过程中才能真正建立起来。

前置知识

本书面向有一定技术背景的读者，但不要求你是某个特定领域的专家。以下按“必需”和“推荐”两个层次列出前置知识，帮助你评估自己的准备程度。

必需：阅读全书的基础

- **Python 编程**：书中几乎所有实验都基于 Python。你需要熟悉 Python 的基本语法、常用的数据结构、包管理（pip）等基本

概念。不要求精通，但应能读懂和修改中等复杂度的 Python 代码。

- **LLM 的基本使用经验**：你应该用过 ChatGPT、Claude 或类似的产品，理解“提示词 (Prompt) → 模型回复”的基本交互模式。
- **一款 AI 辅助编程工具**：强烈建议安装并熟悉至少一款 AI 辅助编程工具，如 Claude Code、Codex、Cursor、TRAE 等。一方面，这些工具能显著提高书中实验的开发效率，因为实验涉及大量代码编写和调试。另一方面，这些编程工具本身就是成熟的 Coding Agent，你在使用它们的过程中会直观地体验到 ReAct 循环、工具调用、上下文管理等书中反复讨论的核心机制。这种第一手体验对理解 Agent 的设计原则极有价值。
- **软件工程常识**：熟悉命令行操作、Git 版本控制、JSON 数据格式、REST API 等基本概念。这些是运行实验和理解 Agent 工具调用机制的基础。

推荐：提升特定章节的阅读体验

- **机器学习基础**（第八章）：了解训练与推理、损失函数、梯度下降、过拟合等基本概念，有助于理解模型后训练。
- **基础数学**（第 2-3、8 章）：对线性代数有直观的理解（比如知道向量可以表示方向和大小、矩阵可以做批量运算）有助于理解嵌入和注意力机制；基本的概率统计知识有助于理解评估指标和强化学习中的期望奖励。书中的数学不涉及复杂的推导，侧重直观解释。
- **Web 开发基础**（第 6 章）：了解 HTTP、WebSocket、前后端分离架构等概念，有助于理解事件驱动的异步 Agent 架构和语音 Agent 的实时通信实验。
- **对 Transformer 架构的基本了解**（第 2、8 章）：Transformer 是当前几乎所有大语言模型的底层架构。对于希望系统地补充大模型基础知识的读者，推荐阅读《图解大模型》(图灵出版)。该书以直观的图解方式讲解了 Transformer 架构、预训练与微调等核心概念，与本书的 Agent 工程视角形成良好的互补。

如果你在某些前置知识上有所欠缺，不必因此却步。本书的核心价值在于**架构设计原则和工程实践的方法论**，而非某个具体的算法或技巧。除第八章后训练以外，全书对数学和机器学习的要求很低，完全可以作为起点。

Agent 技术仍在快速演进，但**好的架构设计原则具有穿越时间的力量**。掌握了“为什么要这样设计”，你就能在技术浪潮的变化中保持清醒的判断力。希望这本书能成为你构建 AI Agent 的可靠指南。

致谢

感谢图灵的梦鸽老师和刘美英老师的辛勤编辑，以及为组织图灵《AI Agent 实战营》课程付出的努力；感谢刘俊明老师在国科大开设 AI Agent 实战课程。也要特别感谢图灵《AI Agent 实战营》的所有学员，以及国科大 AI Agent 实战课程的所有同学——在我讲授这些课程的过程中，大家给了我许多有价值的反馈与建议，也让我对这些概念本身有了更清晰的理解。

感谢 Pine AI 的所有同事。如果没有 Pine AI 这样优秀的产品，以及它所带来的种种挑战，我不可能在 Agent 领域获得如此深入的理解与实践；在一次次思想碰撞中，同事们也提出了许多宝贵的见解。

也要感谢 AI 业界的许多朋友（在此不一一具名）。在各种行业讨论中，大家对我的观点给予了坦诚的反馈，纠正了我不少错误的判断，提升了我对模型与 Agent 的认知。

最要感谢的，是我的家人，特别是我的太太孟佳颖。她始终支持我完成本书的写作，还为本书提出了许多宝贵的意见。